



EMERITUS

Learn. From the world's best.

Model Regularization and Evaluation



Understanding Regression Models

From Basics to Key Challenges in Predictive Modelling

Linear Regression

Basis of predictive models

Multiple Regression

Works with several input features

Polynomial Regression

Captures non-linear trends

Core Challenge

Balancing generalisation and overfitting

Understanding Model Fit

Striking the Right Balance in Machine Learning

Underfitting

High bias and low variance. The model is too simple to learn patterns

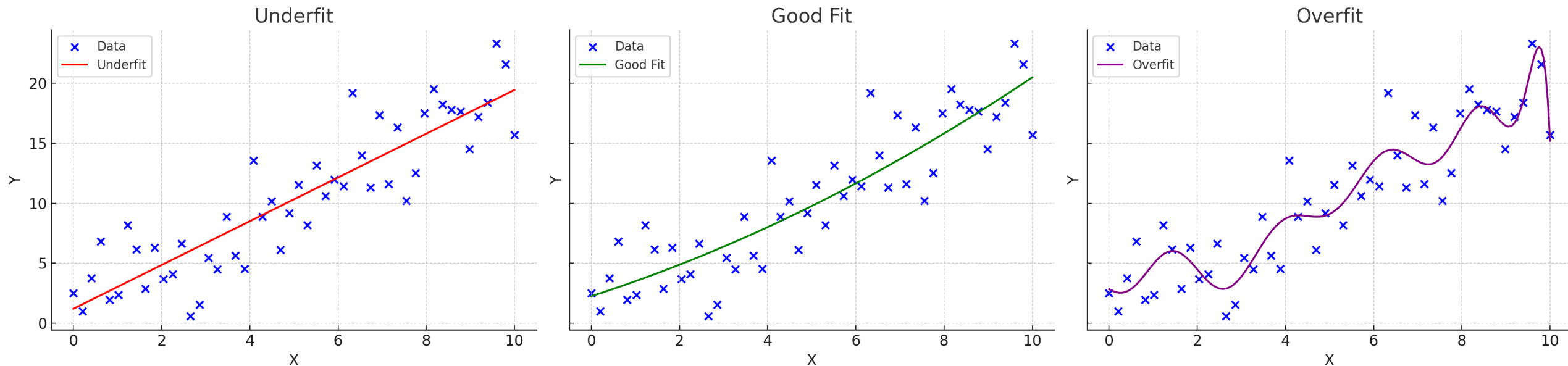
Good Fit

Balanced bias and variance. Performs well on new data

Overfitting

Low bias and high variance. Memorises noise, not patterns

Model Fit Comparison: Underfit vs Good Fit vs Overfit



Regularisation Motivation

Controlling Model Complexity for Better Performance



The Problem:

Complex models pick up noise. Coefficients grow too large.
Poor at handling new data



The Solution:

Add penalties to the loss function. Keep coefficients small.
Encourage simpler models



The Benefits:

Improved generalisation. Less overfitting. More reliable predictions

Regularisation in Regression: Recap

Controlling Model Complexity for Better Performance

Technique	Penalty Type	Effect on Coefficients	Use Case
Ridge	L2: Sum of squares of weights	Shrinks all coefficients	When all features matter
Lasso	L1: Sum of absolute weights	Some coefficients shrink to zero	For feature selection
ElasticNet	Combination of L1 and L2	Mix of shrinkage + sparsity	Best of both worlds

Regularization in Regression: Implementation

Regularisation in Regression

Predicting House Prices in Boston



Dataset

U.S. Census data on Boston housing — includes property and neighbourhood details



Objective

Predict median house price (MEDV) using area features



Why It Matters

Helps in real estate planning, tax decisions, and socio-economic analysis. Real-world use of regression and regularisation techniques.

Boston Housing Dataset – Key Features

Input Variables for House Price Prediction

Feature	Description
CRIM	Per capita crime rate by town
ZN	Proportion of residential land zoned for large lots
INDUS	Proportion of non-retail business acres per town
CHAS	Charles River dummy variable (1 if tract bounds river; 0 otherwise)
NOX	Nitric oxides concentration (parts per 10 million)
RM	Average number of rooms per dwelling
AGE	Proportion of owner-occupied units built prior to 1940
DIS	Weighted distances to five Boston employment centers
RAD	Index of accessibility to radial highways
TAX	Full-value property tax rate per \$10,000
PTRATIO	Pupil-teacher ratio by town
LSTAT	% lower status of the population
HOUSEPRICE (Target)	Median value of owner-occupied homes in \$1000s

Regularisation in Regression

Using Polynomial Models with Ridge and Lasso

```
## Dataset: housingdata.csv
```

Regularisation in Regression – Code Setup

Importing Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

Regularisation in Regression

Load Housing Dataset

```
# Make sure 'housingdata.csv' is in your working directory  
df = pd.read_csv('housingdata.csv')  
df.head()
```

Regularisation in Regression

Step 3: Define the Problem

Objective: Predict 'Price' using one or more features

Data assumed to be non-linear

Apply polynomial regression

Regularisation in Regression

Select Features

Use a single feature (e.g., 'Area') for simplicity

```
X = df.drop(columns=['HOUSEPRICE']) # Independent variable
```

```
y = df['HOUSEPRICE'] # Target variable
```

Regularisation in Regression

Step 4: Split Data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Regularisation in Regression

```
poly_degree = 2  
poly_model = make_pipeline(PolynomialFeatures(degree=poly_degree), Ridge(alpha=0)) # Ridge with  
alpha=0 behaves like no regularization  
poly_model.fit(X_train, y_train)  
y_pred_train = poly_model.predict(X_train)  
y_pred = poly_model.predict(X_test)  
print("Polynomial Regression (No Regularization):")  
print("R2 Score Train:", r2_score(y_train, y_pred_train))  
print("R2 Score Test:", r2_score(y_test, y_pred))  
print("RMSE Train:", np.sqrt(mean_squared_error(y_train, y_pred_train)))  
print("RMSE Test:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Regularisation in Regression

Step 6: Ridge Regression

```
ridge_model = make_pipeline(PolynomialFeatures(degree=poly_degree), Ridge(alpha=1000))
ridge_model.fit(X_train, y_train)
ridge_pred = ridge_model.predict(X_test)
y_pred_train = ridge_model.predict(X_train)
y_pred = ridge_model.predict(X_test)
print("Polynomial Regression (Ridge Regularization):")
print("R2 Score Train:", r2_score(y_train, y_pred_train))
print("R2 Score Test:", r2_score(y_test, y_pred))
print("RMSE Train:", np.sqrt(mean_squared_error(y_train, y_pred_train)))
print("RMSE Test:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Regularisation in Regression

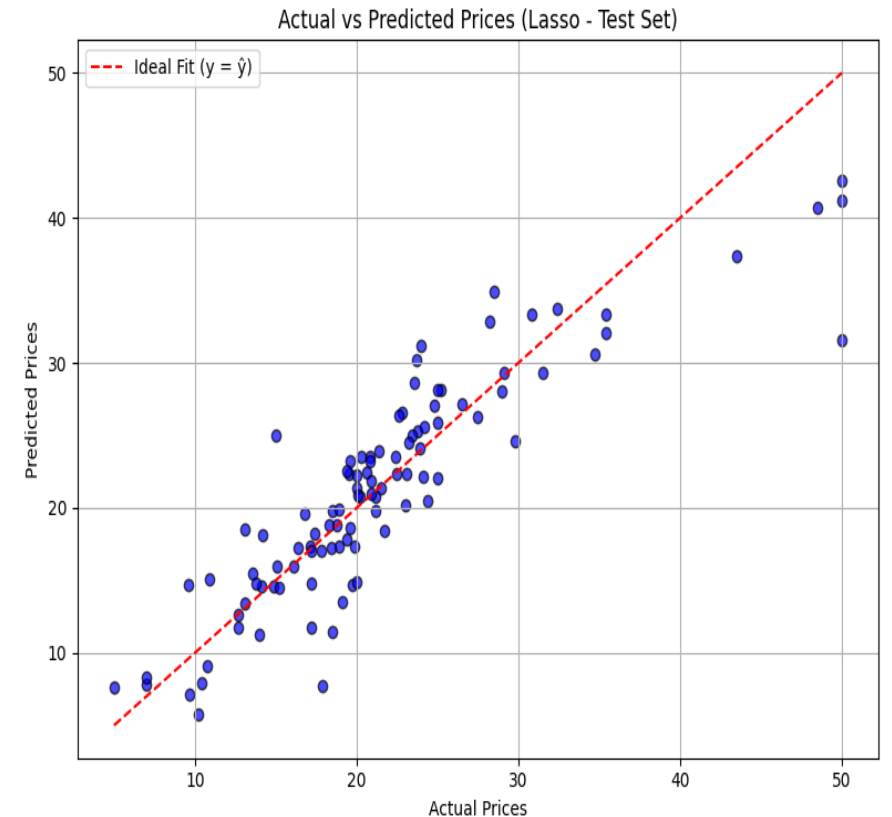
Step 7: Lasso Regression

```
lasso_model = make_pipeline(PolynomialFeatures(degree=poly_degree), Lasso(alpha=10, max_iter=10000))  
lasso_model.fit(X_train, y_train)  
lasso_pred = lasso_model.predict(X_test)  
y_pred_train = lasso_model.predict(X_train)  
y_pred = lasso_model.predict(X_test)  
print("Polynomial Regression (Ridge Regularization):")  
print("R2 Score Train:", r2_score(y_train, y_pred_train))  
print("R2 Score Test:", r2_score(y_test, y_pred))  
print("RMSE Train:", np.sqrt(mean_squared_error(y_train, y_pred_train)))  
print("RMSE Test:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Regularisation in Regression

Actual vs Predicted Visualisation

```
# Generate Actual vs Predicted plot for Test Set
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', edgecolor='k',
            alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', label='Ideal Fit (y
         =  $\hat{y}$ )')
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Prices (Lasso - Test Set)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Model Evaluation in Regression

Why Model Evaluation Matters

A Perfect Fit On Training Data May Fail In Production



Generalisation

How well does the model perform on unseen data



Robustness

Consistent performance across different datasets and conditions



Business Value

Reliable predictions drive better decision-making and outcomes

Common Evaluation Metrics

Key Performance Metrics

Metric	Meaning	Ideal Value
MSE	Mean Squared Error	Lower is better
RMSE	Root MSE	Easier to interpret than MSE
MAE	Mean Absolute Error	Less sensitive to outliers
R^2	Coefficient of Determination	Close to 1

Understanding R^2 Score

How Well does your Model Explain the Data?

1

$R^2 = 0.2$

Poor fit – only 20% of variation explained; data widely scattered

2

$R^2 = 0.7$

Good fit – 70% explained; moderate scatter

3

$R^2 = 0.95$

Excellent fit – 95% explained; points tightly clustered

Model Evaluation Metrics

Check Prediction Accuracy using Key Error Measures

```
from sklearn.metrics import mean_squared_error,  
mean_absolute_error, r2_score  
import numpy as np  
  
y_pred = model.predict(X_test)  
print("MAE:", mean_absolute_error(y_test, y_pred))  
print("MSE:", mean_squared_error(y_test, y_pred))  
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))  
print("R²:", r2_score(y_test, y_pred))
```

Train vs Test Performance

Compare Scores to Check for Overfitting

0.95

Train R^2

Strong training fit

0.72

Test R^2

Weaker test fit

0.23

Gap

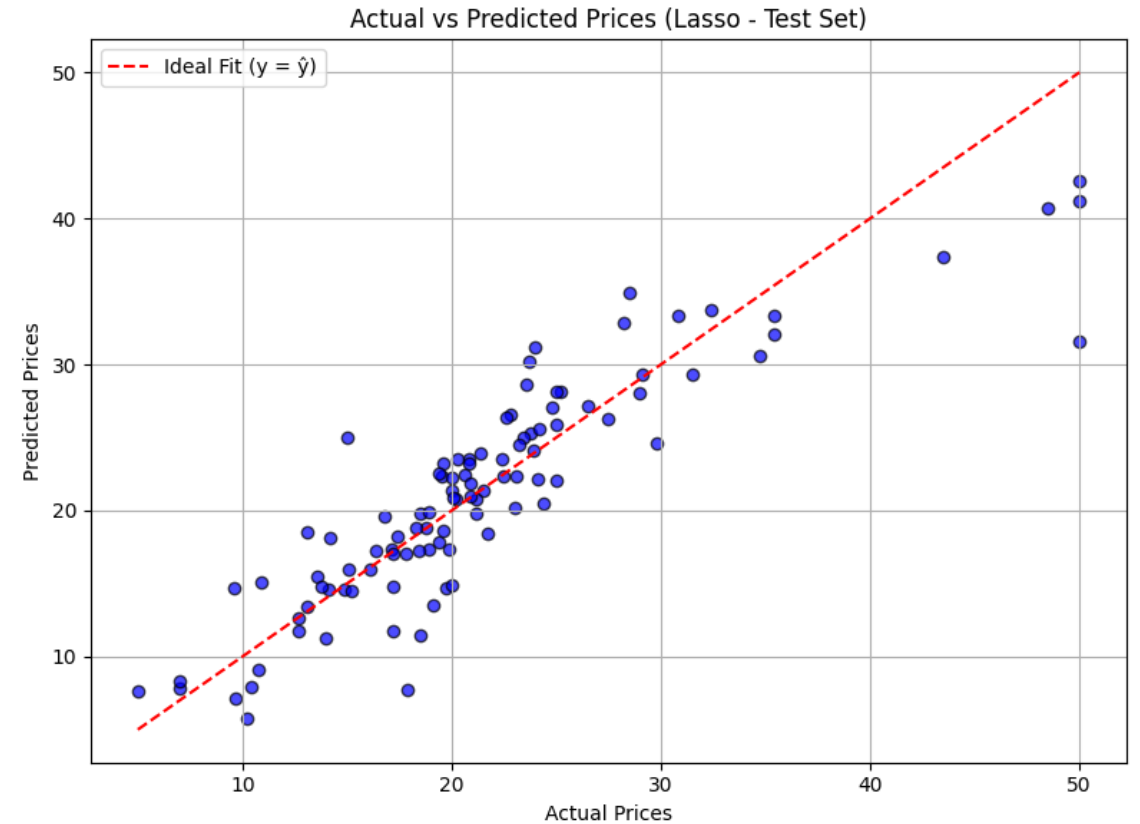
Likely overfitting

Large train-test gaps signal overfitting. Small gaps indicate good generalisation.

Actual vs Predicted Plot

Visual Check of Model Accuracy

- **Perfect fit**
Points fall exactly on diagonal line
- **Good model**
Points cluster tightly around diagonal
- **Poor model**
Wide scatter, points far from line



Key Takeaways

Essential Points for Building and Evaluating Reliable Models



Regularisation

Ridge and Lasso help prevent overfitting



Production

Strong evaluation leads to reliable models



Metrics

R^2 , RMSE, MAE measure model accuracy



Validation

Always check train vs test results

Q&A Session



Open floor for questions and clarifications.